

ERP Escola Profissionalizante

Documentação Técnico-Científica

Políticas de Segurança da Informação - Projeto Integrador

Marcos Antônio Ferreira de Araújo

Matheus de Castro Viana

Repositório: github.com/MatheusdeCastroViana/Project-School-ERP

1. DOCUMENTO DE VISÃO GERAL DO SISTEMA

O sistema consiste em um ERP (Enterprise Resource Planning) voltado para instituições de ensino que oferecem cursos profissionalizantes, com o objetivo de centralizar e integrar os principais processos administrativos, pedagógicos e financeiros da instituição em uma única plataforma.

A arquitetura do sistema foi projetada para contemplar os diferentes setores necessários à gestão da instituição. Nesta primeira versão, estão contemplados os módulos de Gestão e Administração, Pedagógico e Financeiro, com possibilidade de expansão para outros setores e funcionalidades futuramente.

No que se refere à segurança e ao controle de acesso, o sistema contempla mecanismos de autenticação e proteção de dados, incluindo armazenamento seguro de credenciais por meio de funções de hash e salt, recuperação de senha, autenticação de dois fatores (2FA) e registro de logs de autenticação para rastreabilidade das operações relacionadas ao acesso ao sistema.

Além disso, o sistema considera requisitos relacionados à Lei Geral de Proteção de Dados (LGPD), buscando garantir o tratamento adequado dos dados pessoais armazenados e processados pela plataforma, bem como medidas de segurança destinadas à proteção dessas informações.

1.1 Propósito e contexto

Muitas escolas que oferecem cursos profissionalizantes operam por meio de franquias e já dispõem de sistemas informatizados próprios para auxiliar em sua gestão. Entretanto, também existem diversos empreendedores que criam e administram suas próprias escolas, desenvolvendo seus materiais e métodos de ensino, e que igualmente necessitam de uma solução digital para gerenciar funcionários, alunos, aulas, pagamentos, entre outros processos.

Diante desse contexto, o sistema está sendo desenvolvido com foco em alta personalização e flexibilidade, de modo a atender às diferentes necessidades desses empreendedores. O objetivo é disponibilizar uma solução capaz de centralizar as informações e os principais processos da instituição em uma única plataforma, contribuindo para uma gestão mais organizada, integrada e eficiente, além de facilitar a comunicação e a administração interna da escola.

1.1 Escopo do produto

O sistema irá contemplar sete grupos de informação: Institucional (dados da escola, configurações e parâmetros gerais), Pessoas (responsáveis, funcionários, professores e usuários), Acadêmico (cursos, níveis, materiais, turmas, aulas, matrículas, notas e boletins), Financeiro (mensalidades, cobranças, boletos, códigos Pix e pagamentos), Fiscal e Administrativo (controles internos e relatórios institucionais), Operacional (estoque de materiais, jornada de funcionários e cronogramas) e Gestão (indicadores, relatórios automáticos e acompanhamento geral da escola).

1.2 Perfis de acesso

O sistema possui diferentes perfis de acesso, definidos de acordo com os setores e funções desempenhadas pelos usuários dentro da instituição. Os principais setores contemplados são Gestão, Pedagógico e Financeiro/Fiscal, considerados os setores fundamentais para o funcionamento e a administração da escola.

Cada setor é composto por funcionários que possuem cargos e níveis de acesso específicos, de acordo com suas responsabilidades. As permissões de cada usuário são atribuídas conforme sua função, permitindo controlar quais funcionalidades e informações podem ser acessadas ou modificadas.

Os cargos previstos para implementação no sistema são:

- Professor: responsável pelas atividades relacionadas ao ensino, como lançar notas no sistema e gerar relatórios de alunos.
- Administrativo Pedagógico: responsável por atividades administrativas relacionadas ao setor pedagógico.
- Administrativo Financeiro: responsável pelas atividades administrativas e operacionais relacionadas ao setor financeiro.
- Coordenador Pedagógico: responsável pela coordenação e acompanhamento das atividades pedagógicas.
- Coordenador Financeiro: responsável pela coordenação e acompanhamento das atividades financeiras.

- Gestor (Dono da Escola): responsável pela administração geral da instituição, com acesso às informações e funcionalidades necessárias para a gestão do negócio.

1.3 Stack tecnológica

O sistema é desenvolvido em Python 3.14.6, com o framework Django 6.1 e persistência de dados em PostgreSQL 18.6, através do driver psycopg2-binary. A camada de autenticação utiliza django-allauth e django-otp com PyOTP para autenticação de dois fatores (2FA), com geração de QR code via Qrcode e Pillow. As senhas são armazenadas com hash Argon2id, implementado através da biblioteca argon2-cffi. A configuração de ambiente é gerenciada com python-decouple. Atualmente, o backend de e-mail está configurado em modo console (desenvolvimento), sem envio real de mensagens. A integração com um serviço de envio de e-mails será implementada posteriormente.

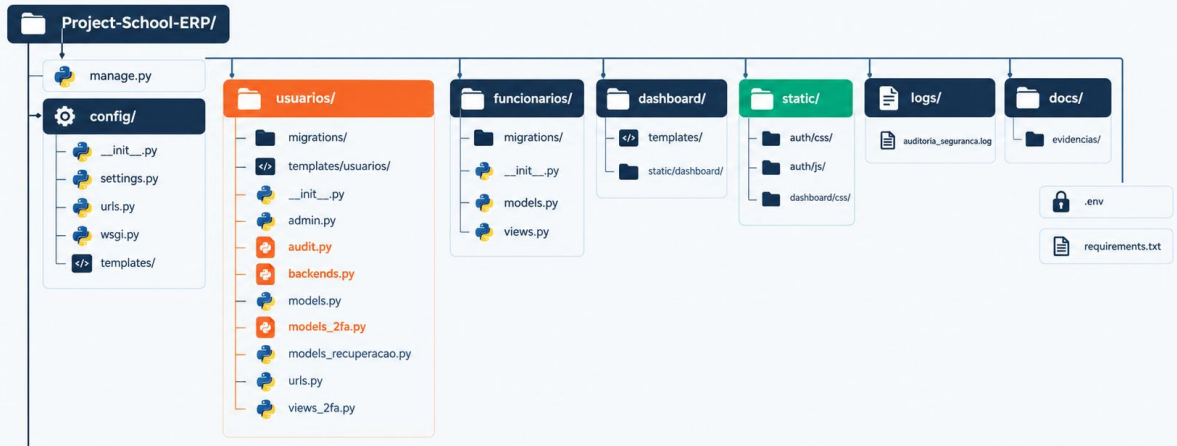
2. DIAGRAMA DE ARQUITETURA

A aplicação é estruturada em um projeto Django que centraliza as configurações e o roteamento principal, distribuindo as requisições entre os apps *usuarios*, *funcionarios* e *dashboard*, além do *allauth* para os fluxos de conta. O app *usuarios* concentra a autenticação, o backend customizado de login, o 2FA, a recuperação de senha e a auditoria de segurança. O app *funcionarios* concentra a estrutura organizacional da escola, através dos modelos *Setor*, *Cargo* e *Funcionario*. O app *dashboard* corresponde à camada de apresentação exibida após o login.

A comunicação entre os apps ocorre principalmente por meio de relacionamentos entre modelos: cada *Usuario* está vinculado a um *Funcionario*, que por sua vez pertence a um Cargo, associado a um Setor e a um conjunto de permissões do Django. Dessa forma, o app *usuarios* depende diretamente do app *funcionarios* para determinar o perfil de acesso de cada usuário autenticado.

A arquitetura segue o padrão MVT (Model-View-Template) do Django: os modelos (models.py de cada app) definem a estrutura dos dados; as views (views.py, views_2fa.py, views_recuperacao.py, entre outras) implementam a lógica de negócio e o controle de acesso; e os templates, organizados por app, compõem a camada de apresentação.





 **Pasta do projeto**
Diretório principal

 **Configuração**
Arquivos de configuração do Django

 **Arquivo Python**
Código fonte (.py)

 **Arquivo de segurança**
Autenticação, 2FA e auditoria

 **Templates**
Arquivos HTML

 **Arquivos estáticos**
CSS, JS, imagens

 **Logs**
Registro de auditoria

 **Variáveis de ambiente**
.env (segredo)

3. FLUXOS DE AUTENTICAÇÃO E DADOS

3.1 Fluxo de login e autenticação

O processo de login é realizado em `/accounts/login/`, por meio do envio de e-mail e senha em formulário protegido por CSRF. O *UsuarioBackend* localiza o usuário pelo e-mail e, em caso de e-mail inexistente ou senha incorreta, retorna `None` sem diferenciar os dois cenários, reduzindo o risco de enumeração de contas.

Antes da validação da senha, é verificado se existe bloqueio ativo decorrente de tentativas consecutivas de autenticação. A cada tentativa inválida, o contador *tentativas_login_falhas* é incrementado e, ao atingir cinco tentativas, a conta é bloqueada por 15 minutos por meio do campo *bloqueado_ate*.

Após a autenticação primária, o *Custom2FAAccountAdapter* intercepta o fluxo do *allauth* por meio do hook *pre_login()*. Quando o usuário possui 2FA habilitado, a autenticação é direcionada para a validação do token TOTP, e a sessão somente é criada após a confirmação do código por *verificar_2fa*.

As sessões expiram após 30 minutos de inatividade. O logout é realizado por requisição POST protegida por CSRF, registra o evento no log de auditoria e, em seguida, invalida a sessão no servidor.

3.2 Fluxo de recuperação de senha

O fluxo de recuperação de senha foi implementado de forma independente do *django-allauth* e é dividido em três etapas. Na solicitação, realizada em `/usuarios/recuperar-senha/`, o usuário informa seu e-mail. Quando o endereço está cadastrado, é gerado um token por meio de *secrets.token_urlsafes(64)*, armazenado no banco na forma de hash SHA-256 e com validade de 24 horas. A resposta apresentada ao usuário é a mesma em ambos os casos, independentemente da existência do e-mail, evitando a enumeração de contas.

Na etapa de validação, realizada em `/usuarios/recuperar-senha/<token>/`, são verificadas a existência, a validade e a utilização prévia do token antes de permitir a definição de uma nova senha. Após a confirmação, o token é marcado como utilizado e não pode ser reutilizado, mesmo que ainda esteja dentro do período de validade.

Todas as etapas do processo são registradas no log de auditoria por meio de *registrar_evento_recuperacao_senha* e *registrar_resultado_recuperacao_senha*. Por questões de segurança, os registros não armazenam o token completo nem a senha do usuário.

3.3 Fluxo de dados pessoais (coleta, uso, exclusão)

Atualmente, o fluxo de dados contempla apenas informações de Funcionários/Usuários, enquanto os módulos de Aluno e Responsável ainda não foram implementados. No cadastro, são coletados somente os dados necessários para identificação, controle de acesso e comunicação: nome, CPF, telefone, cargo e setor.

O consentimento é apresentado ao usuário por meio de um popup no primeiro acesso e controlado por versão (*CONSENT_VERSION*). O registro do aceite é armazenado no localStorage e, quando o usuário está autenticado, também no log de auditoria por meio de uma API específica(*registrar_consentimento_view*).

O sistema disponibiliza ao titular recursos para consulta (*/usuarios/meus-dados/*), exportação dos dados em JSON (*/usuarios/meus-dados/exportar/*) e solicitação de exclusão (*/usuarios/meus-dados/exclusao/*). A exclusão é parcial: o telefone é removido automaticamente, enquanto dados como CPF, e-mail e cargo são mantidos quando necessários para identificação e controle de acesso. As exceções são apresentadas ao titular antes da confirmação da solicitação.

4. GESTÃO DE CREDENCIAIS

A gestão de credenciais no ERP Escola Profissionalizante foi pensada para cobrir todo o ciclo de vida de uma credencial, desde a criação até a invalidação, passando pelo armazenamento e pela recuperação. O ponto de partida foi a decisão de não usar o campo username padrão do Django, substituindo-o pelo e-mail como identificador único do usuário. Essa escolha reduz a superfície de ataque porque elimina a possibilidade de um atacante tentar adivinhar nomes de usuário curtos ou padrões comuns, e ainda alinha o sistema ao comportamento esperado em aplicações corporativas atuais.

No momento do cadastro, a senha passa por uma cadeia de validações antes mesmo de chegar ao banco. O Django aplica, por padrão, quatro validadores: verificação de similaridade com atributos do usuário (como o próprio e-mail), tamanho mínimo, comparação com uma lista de senhas comuns e rejeição de senhas puramente numéricas. Essa camada inicial evita que o usuário escolha credenciais fracas por descuido, sem impor regras excessivas que costumam levar ao efeito colateral de anotar a senha em papel.

O armazenamento da senha utiliza o algoritmo Argon2id. O Argon2id foi escolhido por ser o vencedor do Password Hashing Competition e por combinar resistência a ataques de GPU (característica do Argon2d) com resistência a ataques de canal lateral (característica do Argon2i). Cada senha recebe um salt único gerado automaticamente pelo próprio Django, o que impede o uso de tabelas rainbow pré-computadas. O hash resultante é armazenado no banco no formato padrão do Django, que inclui o identificador do algoritmo, os parâmetros, o salt e o hash propriamente dito, isso permite, no futuro, migrar para um algoritmo mais forte sem invalidar as senhas existentes.

Para o segundo fator de autenticação, o sistema utiliza TOTP (Time-based One-Time Password) conforme definido na RFC 6238, implementado via biblioteca pyotp. Cada usuário que ativa o 2FA recebe uma chave secreta em base32, armazenada no modelo Configuracao2FA em relacionamento um-para-um com o usuário. A chave é usada para gerar códigos de seis dígitos que se renovam a cada 30 segundos. O fluxo foi desenhado para que o 2FA seja validado somente após a autenticação primária ter sido bem-sucedida, evitando que um atacante explore a verificação do segundo fator para enumerar contas ou contornar o bloqueio por força bruta. A sessão só é efetivamente criada após a confirmação do código TOTP, por meio de um adapter customizado do django-allauth que intercepta o hook `pre_login`.

A recuperação de senha segue um fluxo independente do allauth, com token gerado por `secrets.token_urlsafe(64)` e armazenado no banco na forma de hash SHA-256. O token tem validade de 24 horas e é invalidado imediatamente após o uso bem-sucedido, impedindo reutilização. A mensagem de resposta é a mesma independentemente de o e-mail estar ou não cadastrado, o que mitiga ataques de enumeração de contas.

As sessões autenticadas expiram após 30 minutos de inatividade, com renovação automática a cada requisição. O logout é implementado via POST protegido por CSRF, registra o evento no log de auditoria e só então invalida a sessão no servidor, impedindo a reutilização do cookie `sessionid`. Por fim, o sistema aplica bloqueio temporário de 15 minutos após cinco tentativas consecutivas de login com senha incorreta, registrando cada tentativa no log de auditoria com o endereço IP de origem.

5. USO DE CRIPTOGRAFIA

6. IDENTIFICAÇÃO DOS ATIVOS DO SISTEMA

- ⊄ Dados pessoais de titulares: informações de identificação (nome, CPF), contato (telefone, e-mail) e dados profissionais (cargo, setor, jornada) de funcionários, conforme documentado no inventário de dados pessoais (*docs/lgpd/inventario_dados.md*). Com a implementação futura dos módulos Acadêmico e Financeiro, também passarão a ser protegidos os dados pessoais e financeiros de alunos e responsáveis.
- ⊄ Credenciais de acesso: e-mail e senha (armazenada com hash Argon2id), segredos de autenticação de dois fatores (TOTP) e tokens de recuperação de senha.
- ⊄ Logs de auditoria e segurança: registros de eventos de autenticação, tentativas de login, uso de 2FA e solicitações relacionadas à LGPD, contendo IP, data/hora e identificação do usuário responsável.
- ⊄ Banco de dados: instância PostgreSQL onde são persistidos todos os dados acima.
- ⊄ Código-fonte da aplicação: em especial os módulos de autenticação, hashing de senha e auditoria, cuja exposição indevida poderia revelar a lógica de segurança implementada.
- ⊄ Infraestrutura de execução: ambiente onde a aplicação Django roda, incluindo variáveis de ambiente (.env) que armazenam segredos de configuração e conexão com o banco.

7. IDENTIFICAÇÃO DE AMEAÇAS E VULNERABILIDADES

As principais possíveis ameaças e vulnerabilidades identificadas no sistema, organizadas por ativo, são:

- ⊄ Credenciais de acesso (senha): ataques de força bruta ou dicionário contra o login; exposição das senhas em caso de vazamento do banco de dados; e reutilização da sessão após logout ou expiração.
- ⊄ Segredo TOTP (2FA): interceptação ou exposição da chave secreta durante a ativação do 2FA; e ataques de phishing para obtenção dos códigos TOTP.
- ⊄ Token de recuperação de senha: interceptação ou tentativa de adivinhação do token; e identificação de tokens válidos por tentativas sucessivas.
- ⊄ Dados pessoais (CPF, nome e telefone): acesso indevido por usuários sem necessidade funcional; e exposição decorrente de falhas na aplicação, como injeção de SQL.
- ⊄ Logs de auditoria: alteração ou exclusão dos registros para ocultar atividades maliciosas; e exposição de informações sensíveis armazenadas nos logs.
- ⊄ Banco de dados PostgreSQL: acesso não autorizado devido ao uso de credenciais fracas, padrão ou expostas.
- ⊄ Código-fonte da aplicação: exposição pública do repositório, possibilitando a análise da lógica da aplicação e identificação de informações sensíveis.
- ⊄ Infraestrutura de execução: comprometimento do servidor, podendo resultar na exposição de variáveis de ambiente, credenciais e outros segredos.
- ⊄ Configuração da aplicação: utilização de configurações de desenvolvimento em ambiente de produção, como `DEBUG=True` e `ALLOWED_HOSTS` vazio, aumentando a exposição do sistema.

8. ASSOCIAÇÃO RISCO X CONTRAMEDIDA

A tabela abaixo relaciona os ativos identificados na seção anterior com as principais ameaças e vulnerabilidades associadas, as contramedidas implementadas no sistema e uma avaliação qualitativa do risco residual após a aplicação dos controles. A classificação do risco residual segue a escala baixa, média ou alta, considerando a probabilidade de ocorrência e o impacto potencial para a instituição.

Ativo	Ameaça / vulnerabilidade	Contramedida adotada	Risco residual
Credenciais de acesso (senha)	Ataque de força bruta ou dicionário contra o endpoint de login.	Bloqueio temporário após 5 tentativas; hash Argon2id com salt único; validadores de complexidade no cadastro.	Baixo
Credenciais de acesso (senha)	Vazamento do banco de dados expõe senhas em texto claro.	Senhas nunca são armazenadas em texto puro; apenas o hash Argon2id com metadados é persistido.	Baixo
Credenciais de acesso (senha)	Reutilização de sessão após logout ou expiração.	Invalidação server-side da sessão no logout; expiração por inatividade de 30 minutos; cookie HttpOnly e SameSite=Lax.	Baixo
Segredo TOTP (2FA)	Interceptação da chave secreta durante a ativação.	Chave transmitida apenas no momento da ativação, em canal HTTPS (em produção); QR Code exibido uma única vez.	Baixo
Segredo TOTP (2FA)	Ataque de phishing para capturar código TOTP.	Código TOTP tem validade de 30 segundos; janela de tolerância de apenas 1 período; logs de verificação registrados.	Médio
Token de recuperação de senha	Token interceptado ou adivinhado permite redefinição de senha.	Token gerado com secrets.token_urlsaf(64) e armazenado como hash SHA-256; validade de 24 horas; invalidação após uso.	Baixo

Token de recuperação de senha	Enumeração de tokens válidos por tentativa e erro.	Espaço de busca de 256 bits torna a enumeração impraticável; mensagens de erro genéricas não revelam existência do token.	Baixo
Dados pessoais (CPF, nome, telefone)	Acesso indevido por usuário autenticado sem necessidade funcional.	Controle de acesso baseado em cargo e setor; princípio do menor privilégio aplicado nas views.	Médio
Dados pessoais (CPF, nome, telefone)	Vazamento por falha na aplicação (ex.: SQL Injection).	Uso de ORM Django com parâmetros; validação de entrada; proteção CSRF habilitada globalmente.	Baixo
Logs de auditoria	Alteração ou exclusão para encobrir atividade maliciosa.	Permissões de admin bloqueiam adição, edição e exclusão de logs; apenas superusuário pode excluir, e mesmo assim com registro prévio.	Baixo
Logs de auditoria	Exposição de dados sensíveis (senha, token) nos próprios logs.	Funções de auditoria registram apenas o evento e o resultado, nunca o segredo em si (RN-10).	Baixo
Banco de dados PostgreSQL	Acesso direto por credenciais fracas ou padrão.	Credenciais armazenadas em variáveis de ambiente (.env), não no código; usuário dedicado com privilégios mínimos.	Médio
Código-fonte da aplicação	Exposição pública revela lógica de segurança.	Repositório privado; .env e logs/ no .gitignore; chaves criptográficas fora do código.	Baixo
Infraestrutura de execução	Comprometimento do servidor expõe variáveis de ambiente.	Segredos isolados em .env; em produção, recomenda-se uso de gerenciador de segredos e TLS obrigatório.	Médio

A avaliação de risco residual considera que algumas ameaças, como o phishing de código TOTP ou o acesso indevido por usuário legítimo com privilégios excessivos, não podem ser eliminadas apenas por controles técnicos elas dependem também de conscientização dos usuários e de revisão periódica de permissões pela gestão da escola. Nesses casos, o risco foi classificado como médio mesmo após a aplicação das contramedidas, indicando a necessidade de controles complementares de natureza organizacional.

9. TESTES DE SEGURANÇA REALIZADOS

A camada de autenticação e gestão de credenciais foi validada por meio de testes automatizados e manuais. Os testes automatizados, executados com *python manage.py test usuarios*, verificam o uso do Argon2id e seus parâmetros de custo, a autenticação com credenciais válidas e inválidas e o fluxo de recuperação de senha, incluindo geração, registro e validação de tokens.

Os testes manuais realizados pelo front-end verificaram o login, o bloqueio após cinco tentativas incorretas, a ativação do 2FA por QR Code, a geração de salt individual, a recuperação de senha e o registro de eventos no log de auditoria, com prints utilizados como evidência. O fluxo de logout também foi validado com o *Client* de testes do Django, verificando o encerramento da sessão e a restrição de acesso após o logout.

10. RESULTADOS DOS TESTES

Todos os testes descritos a seguir podem ser executados em conjunto por meio do comando *python manage.py test usuarios*. As evidências dos testes manuais estão disponíveis no diretório *docs/evidencias*.

10.1 Hash de senha e autenticação

Teste realizado	Ferramenta / Método	Resultado / Evidência
Hash de senha usa Argon2id	UsuarioArgon2PasswordHasherTests.test_hash_usa_argon2id	Passou (OK)
Parâmetros de custo do hash (t=5, m=32768, p=1)	UsuarioArgon2PasswordHasherTests.test_parametros_de_custo_configurados	Passou (OK)
Autenticação com senha correta	UsuarioArgon2PasswordHasherTests.test_senha_correta_autentica	Passou (OK)
Autenticação com senha incorreta	UsuarioArgon2PasswordHasherTests.test_senha_incorreta_nao_autentica	Passou (OK)
Tempo de hash dentro do esperado (< 2s)	UsuarioArgon2PasswordHasherTests.test_tempo_de_hash_dentro_do_esperado	Passou (OK) Aproximadamente 92ms

10.2 Recuperação de senha

Teste realizado	Ferramenta / Método	Resultado / Evidência
Solicitação gera token para e-mail existente	RecuperacaoSenhaTests.test_solicitacao_gera_token_para_email_existente	Passou (OK) Console exibe o bloco com token, data de expiração e link de recuperação
Solicitação com e-mail inexistente não gera token	RecuperacaoSenhaTests.test_solicitacao_com_email_inexistente_nao_gera_token	Passou (OK)
Log registrado ao encontrar o e-mail	RecuperacaoSenhaTests.test_log_registrado_na_solicitacao_email_encontrado	Passou (OK)
Log registrado ao não encontrar o e-mail	RecuperacaoSenhaTests.test_log_registrado_na_solicitacao_email_nao_encontrado	Passou (OK)
Token válido permite trocar a senha	RecuperacaoSenhaTests.test_token_valido_permite_trocar_senha_e_logar_sucesso	Passou (OK)
Token reutilizado é recusado	RecuperacaoSenhaTests.test_token_reutilizado_falha_e_logar	Passou (OK)
Token expirado é recusado	RecuperacaoSenhaTests.test_token_expirado_falha_e_logar	Passou (OK)

Token inexistente é recusado	RecuperacaoSenhaTests.test_token_inexistente_logar_falha	Passou (OK)
---------------------------------	--	-------------

11. REFERÊNCIAS

OWASP FOUNDATION. **Password Storage Cheat Sheet**. OWASP Cheat Sheet Series, 2023. Disponível em: https://cheatsheetseries.owasp.org/cheatsheets/Password_Storage_Cheat_Sheet.html. Acesso em: 29 out. 2026.

ABNT. NBR ISO/IEC 27001:2022 - Tecnologia da informação - Técnicas de segurança - Sistemas de gestão de segurança da informação - Requisitos. Rio de Janeiro: ABNT, 2022.

ABNT. NBR ISO/IEC 27002:2022 - Tecnologia da informação - Técnicas de segurança - Código de prática para controles de segurança da informação. Rio de Janeiro: ABNT, 2022.

ANPD. Guia de Boas Práticas para Tratamento de Dados Pessoais. Autoridade Nacional de Proteção de Dados, 2024. Disponível em: <https://www.gov.br/anpd/pt-br>. Acesso em: 27 set. 2026.

BRASIL. Lei nº 13.709, de 14 de agosto de 2018. Dispõe sobre a proteção de dados pessoais e altera a Lei nº 12.965, de 23 de abril de 2014 (Marco Civil da Internet). Diário Oficial da União, Brasília, DF, 15 ago. 2018.

MONTEIRO, U. A. Criptografia, Segurança de Redes e Sistemas. Rio de Janeiro: Campus, 2012.

12. RASTREABILIDADE E CONTROLE DE MUDANÇAS

Versão	Data	Alteração	Responsável
1.0	27/09/2026	Primeira versão da documentação	Marcos Antônio Ferreira de Araújo e Matheus de Castro Viana